Chapter 3 — Functions and Expressions

Chapter 2 selected and filtered stored values. Real reports need to *transform* them: round a price, tidy a messy email, work out how many days an order took to ship, label a product "premium", or show "no email on file" instead of a blank. This chapter covers the building blocks for all of that: SQL's **type system** and conversions, **number**, **string** and **date/time** functions, and the **conditional expressions** `CASE`, `COALESCE`, `NULLIF`, `GREATEST` and `LEAST`.

The SQL course these notes draw on was written for MySQL. Function names differ more between databases than anything else in SQL, so every section ends with a **dialect map** from MySQL (and SQL Server, SQLite) to PostgreSQL.

What you'll learn

#	Topic	You will be able to…
3.1	Types and casts	name PostgreSQL's core types, convert with `CAST` / `::`, avoid float surprises
3.2	Number functions	round, truncate, take powers/logs/roots, generate random numbers, format numbers
3.3	String functions	join, cut, measure, clean, search, pad, split and pattern-replace text
3.4	Date and time functions	get "now", do date arithmetic, extract/truncate parts, format/parse, handle time zones
3.5	Conditional expressions	use `CASE`, `COALESCE`, `NULLIF`, `GREATEST` / `LEAST`
3.6	Worked exercises	combine functions into realistic report columns

Setup. Examples use the `company` sample database (chapter 1, 1.11); code is in `code/`. Outputs come from PostgreSQL 14.

Functions in one paragraph

A **function** takes zero or more **arguments** and returns one value: `round(9.876, 1)` → `9.9`. Functions that work on one row at a time (everything in this chapter) are **scalar functions**; functions that squash many rows into one (`sum`, `count`, `avg`) are **aggregate functions**, covered in chapter 4. Functions nest — `lower(trim(email))` — and can appear anywhere an expression can: the `SELECT` list, `WHERE`, `ORDER BY`, `GROUP BY`, `UPDATE … SET`.

3.1 Data types and conversions

What it is

Every value in SQL has a **data type** that decides what it can hold and which operators and functions apply. `'42'` (text) and `42` (integer) look alike but behave differently: `'42' || '1'` is `'421'`, `42 + 1` is `43`. PostgreSQL is **strict**: it rarely converts types silently, which catches bugs early.

The core PostgreSQL types

Family	Type	Holds	Notes
Integer	`smallint`, `integer` (`int`), `bigint`	2-, 4-, 8-byte whole numbers	`integer` max is 2,147,483,647
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	exact numbers, *p* total digits, *s* after the point	use for **money**; slower than floats
Floating point	`real`, `double precision`	approximate binary fractions	science/measurements, never money
Text	`text`, `varchar(n)`, `char(n)`	strings	in PostgreSQL `text` and `varchar` perform the same; `char(n)` pads with spaces
Boolean	`boolean`	`true`, `false`, `NULL`	
Date/time	`date`, `time`, `timestamp`, `timestamptz`, `interval`	calendar dates, clock times, instants, durations	3.4
Other	`uuid`, `json`/`jsonb`, arrays (`int[]`), `bytea`, enums	uuid = random 128-bit ID; jsonb = stored JSON; array = list in one cell; bytea = raw bytes; enum = fixed set of labels	JSON and arrays in chapter 14

Concept	PostgreSQL	MySQL	SQL Server	SQLite
Exact decimal	`numeric(10,2)`	`DECIMAL(10,2)`	`DECIMAL(10,2)`	stored as REAL/INTEGER (type *affinity*)
Variable text	`text` / `varchar(n)`	`VARCHAR(n)`, `TEXT`	`VARCHAR(n)` / `NVARCHAR(n)`	`TEXT`
Boolean	`boolean`	`TINYINT(1)` alias `BOOLEAN`	`BIT`	integer 0/1
Timestamp with zone	`timestamptz`	`TIMESTAMP` (converted to UTC, the world reference time)	`DATETIMEOFFSET`	text or number, by convention

(SQLite is the outlier: a column's type is only a *preference* — it will store text in an integer column. The full DDL (Data Definition Language — the `CREATE`/`ALTER` statements that define tables) story is in chapter 9.)

How it works — asking for and changing types

`pg_typeof(x)` reports a value's type:

```
-- code/01_types_and_casts.sql
SELECT pg_typeof(42) AS int_lit, pg_typeof(42.0) AS dec_lit, pg_typeof(4.2e1) AS sci_lit,
       pg_typeof('hi') AS str_lit, pg_typeof(true) AS bool_lit, pg_typeof(now()) AS now_type;
SELECT pg_typeof(salary) AS salary, pg_typeof(hire_date) AS hire_date, pg_typeof(email) AS email
FROM employees LIMIT 1;
```

Output:

```
 int_lit | dec_lit | sci_lit | str_lit | bool_lit |         now_type
---------+---------+---------+---------+----------+--------------------------
 integer | numeric | numeric | unknown | boolean  | timestamp with time zone
(1 row)

 salary  | hire_date | email
---------+-----------+-------
 numeric | date      | text
(1 row)
```

Notice: `42.0` is `numeric` (exact), not a float, and a quoted literal starts out as type `unknown` — PostgreSQL decides its type from context. That's why `WHERE hire_date >= '2023-01-01'` works: the string is read as a `date` because it's compared with one.

Casting (explicit conversion) has three spellings:

Form	Standard?	Example
`CAST(expr AS type)`	yes — works everywhere	`CAST('2025-03-18' AS date)`
`expr::type`	PostgreSQL shorthand	`'2025-03-18'::date`
`type 'literal'`	standard, literals only	`date '2025-03-18'`

```
SELECT CAST('2025-03-18' AS date) AS std_cast, '2025-03-18'::date AS pg_cast, date '2025-03-18' AS typed_literal;
SELECT CAST(100 AS numeric(5,2)) AS to_numeric, 3.99::integer AS rounds_not_truncates,
       '42'::integer + 1 AS text_to_int, 123::text || 'abc' AS int_to_text;
```

Output:

```
  std_cast  |  pg_cast   | typed_literal
------------+------------+--------------
 2025-03-18 | 2025-03-18 | 2025-03-18
(1 row)

 to_numeric | rounds_not_truncates | text_to_int | int_to_text
------------+----------------------+-------------+-------------
     100.00 |                    4 |          43 | 123abc
(1 row)
```

Casting a decimal to `integer` **rounds** (3.99 → 4); use `trunc()` or `floor()` if you want to cut off the fraction.

Exact vs approximate numbers

`double precision` stores binary fractions, and most decimal fractions (like 0.1) have no exact binary form:

```
SELECT 0.1 + 0.2 = 0.3 AS numeric_equal,
       0.1::double precision + 0.2::double precision = 0.3 AS float_equal,
       0.1::double precision + 0.2::double precision AS float_sum;
SELECT 12.345::numeric(5,2) AS rounded_to_scale;
```

Output:

```
 numeric_equal | float_equal |      float_sum
---------------+-------------+---------------------
 t             | f           | 0.30000000000000004
(1 row)

 rounded_to_scale
------------------
            12.35
(1 row)
```

Rule: **money and anything that must add up exactly → `numeric`**. A `numeric(p,s)` value is rounded to *s* decimal places when stored.

Errors a strict type system gives you

```
SELECT 2147483647::integer + 1;
SELECT 'abc'::integer;
SELECT 1000::numeric(5,2);
SELECT first_name + 1 FROM employees LIMIT 1;
```

Output:

```
ERROR:  integer out of range
ERROR:  invalid input syntax for type integer: "abc"
LINE 1: SELECT 'abc'::integer;
               ^
ERROR:  numeric field overflow
DETAIL:  A field with precision 5, scale 2 must round to an absolute value less than 10^3.
ERROR:  operator does not exist: text + integer
LINE 1: SELECT first_name + 1 FROM employees LIMIT 1;
                          ^
HINT:  No operator matches the given name and argument types. You might need to add explicit type casts.
```

MySQL would quietly turn `'abc' + 1` into `1` (with a warning) and wrap or clip some overflows (values too big for their type) depending on its SQL mode (a server setting that controls how strict MySQL is). PostgreSQL refuses — annoying for a minute, valuable for years.

Booleans accept several spellings on input:

```sql
SELECT 'yes'::boolean AS yes, 'f'::boolean AS f, 'on'::boolean AS on_;
```

```
 yes | f | on_
-----+---+-----
 t   | f | t
(1 row)
```

Common mistakes

- Storing money in `real` / `double precision`.
- Storing numbers or dates in text columns — sorting and comparisons become alphabetical (`'10' < '9'`).
- Expecting `::integer` to truncate; it rounds.

Self-check

1. *Type of the literal `42.0` in PostgreSQL?* → `numeric`.
2. *Portable way to convert text to a date?* → `CAST('2025-03-18' AS date)`.
3. *Why is `0.1::float8 + 0.2::float8 = 0.3` false?* → Binary floating point can't represent 0.1 or 0.2 exactly.

TL;DR

Everything has a type. Use `numeric` for money, `text` for strings, `timestamptz` for instants. Convert with `CAST(x AS t)` or `x::t`. PostgreSQL won't silently coerce (automatically convert) mismatched types — that's a feature.

3.2 Number functions

What it is

Numeric functions do arithmetic you'd otherwise hand-code: rounding for display, absolute differences, powers and logarithms for growth (a **logarithm** answers "to what power must I raise the base to get this number?" — log base 10 of 1000 is 3), modulo for "every n-th", random numbers for sampling.

The functions

Function (PostgreSQL)	Does	Example → result
`abs(x)`	absolute value	`abs(-10)` → 10
`round(x)` / `round(x, d)`	round to *d* decimals (negative *d* rounds left of the point)	`round(9.876, 1)` → 9.9; `round(1234.567, -2)` → 1200
`trunc(x, d)`	cut to *d* decimals, no rounding	`trunc(9.876, 1)` → 9.8
`ceil(x)` / `ceiling(x)`	smallest integer ≥ x (round **up**)	`ceil(4.3)` → 5; `ceil(-4.3)` → -4
`floor(x)`	largest integer ≤ x (round **down**)	`floor(4.8)` → 4; `floor(-4.3)` → -5
`sqrt(x)`, `cbrt(x)`	square / cube root	`sqrt(25)` → 5
`power(a, b)`, `a ^ b`	a to the power b	`power(2, 3)` → 8
`mod(a, b)`, `a % b`	remainder	`mod(17, 5)` → 2
`div(a, b)`	integer quotient	`div(17, 5)` → 3
`exp(x)`	e^x (e ≈ 2.718, the natural growth constant)	`exp(2)` → 7.389…
`ln(x)`	natural log (base e)	`ln(10)` → 2.302585…
`log(x)`, `log10(x)`	**base-10** log in PostgreSQL	`log(100)` → 2
`log(b, x)`	log of x in base b	`log(2, 8)` → 3
`sign(x)`	-1, 0 or 1	`sign(-15)` → -1
`pi()`, `degrees(r)`, `radians(d)`	π and angle conversion	`radians(60)` → 1.047…
`greatest(…)`, `least(…)`	largest / smallest argument	`greatest(3, 9, 4)` → 9
`gcd(a, b)`, `lcm(a, b)`	greatest common divisor / least common multiple (PostgreSQL 13+)	`gcd(12, 18)` → 6
`random()`, `setseed(s)`	random `double precision` in [0, 1) (0 included, 1 excluded); seed it (give a fixed starting value) for repeatable output	
`to_char(x, fmt)`	format a number as text	`to_char(1234567, 'FM9,999,999.00')` → `1,234,567.00`

How it works — run them

```sql
-- code/02_number_functions.sql
SELECT abs(-10) AS abs, round(9.876, 1) AS round_1, round(9.876) AS round_0,
       round(1234.567, -2) AS round_neg, trunc(9.876, 1) AS trunc_1,
       ceil(4.3) AS ceil, floor(4.8) AS floor, ceil(-4.3) AS ceil_neg, floor(-4.3) AS floor_neg;
```

Output:

```
 abs | round_1 | round_0 | round_neg | trunc_1 | ceil | floor | ceil_neg | floor_neg
-----+---------+---------+-----------+---------+------+-------+----------+-----------
  10 |     9.9 |      10 |      1200 |     9.8 |    5 |     4 |       -4 |        -5
(1 row)
```

With negative numbers `ceil` moves toward zero and `floor` away from it — `floor(-4.3)` is -5, not -4. `trunc` always moves toward zero.

Halfway cases depend on the type:

```
SELECT round(2.5) AS num_2_5, round(-2.5) AS num_neg_2_5,
       round(2.5::double precision) AS dbl_2_5, round(3.5::double precision) AS dbl_3_5;
```

Output:

```
 num_2_5 | num_neg_2_5 | dbl_2_5 | dbl_3_5
---------+-------------+---------+---------
       3 |          -3 |       2 |       4
(1 row)
```

`numeric` rounds halves **away from zero** (2.5 → 3); `double precision` typically rounds halves **to the nearest even** number ("banker's rounding", 2.5 → 2, 3.5 → 4) because it uses the platform's floating-point rounding. Another reason to keep money in `numeric`.

```
SELECT sqrt(25) AS sqrt, power(2, 3) AS power, 2 ^ 3 AS caret, mod(17, 5) AS mod,
       17 % 5 AS pct, mod(-17, 5) AS mod_neg, div(17, 5) AS int_div;
SELECT exp(2) AS exp_2, ln(10) AS ln_10, log(100) AS log_100, log10(100) AS log10_100,
       log(2, 8) AS log_base2_of_8;
SELECT sign(-15) AS sign, pi() AS pi, degrees(1.047) AS degrees, radians(60) AS radians;
SELECT greatest(3, 9, 4) AS greatest, least(3, 9, 4) AS least, gcd(12, 18) AS gcd, lcm(4, 6) AS lcm;
```

Output:

```
 sqrt | power | caret | mod | pct | mod_neg | int_div
------+-------+-------+-----+-----+---------+---------
    5 |     8 |     8 |   2 |   2 |      -2 |       3
(1 row)

      exp_2       |       ln_10       | log_100 | log10_100 |   log_base2_of_8
------------------+-------------------+---------+-----------+--------------------
 7.38905609893065 | 2.302585092994046 |       2 |         2 | 3.0000000000000000
(1 row)

 sign |        pi         |      degrees      |      radians
------+-------------------+-------------------+--------------------
   -1 | 3.141592653589793 | 59.98868115019719 | 1.0471975511965976
(1 row)

 greatest | least | gcd | lcm
----------+-------+-----+-----
        9 |     3 |   6 |  12
(1 row)
```

Two things to remember: the remainder keeps the sign of the **dividend** (the number being divided; `mod(-17, 5)` = -2), and `log(x)` means **base 10** in PostgreSQL but **natural log** in MySQL and SQL Server. Use `ln()` when you mean natural log — it's unambiguous everywhere that has it.

Random numbers

`random()` returns a new value each call. `setseed()` makes the sequence repeatable for the rest of the session (your current connection to the database) — useful for tests and reproducible samples.

```
SELECT setseed(0.42);
SELECT round(random()::numeric, 4) AS r1, round(random()::numeric, 4) AS r2;
SELECT floor(random() * 6)::int + 1 AS dice_roll;
```

Output:

```
   r1   |   r2
--------+--------
 0.5699 | 0.2954
(1 row)

 dice_roll
-----------
         6
(1 row)
```

`floor(random() * n)::int + 1` gives a whole number from 1 to *n*. (Sampling rows randomly — `ORDER BY random() LIMIT k` or `TABLESAMPLE` — comes up in chapter 12.)

Formatting numbers for people

`to_char(number, pattern)` turns numbers into display strings. `9` is a digit position, `0` a digit that's always shown, `,` a group separator, `.` the decimal point, and `FM` ("fill mode") strips the padding spaces `to_char` otherwise adds.

```sql
SELECT to_char(1234567, 'FM9,999,999.00') AS with_commas,
       to_char(0.0725, 'FM990.00%') AS naive_pct,
       to_char(0.0725 * 100, 'FM990.00') || '%' AS pct,
       to_char(42, '00000') AS zero_padded;
```

Output:

```
 with_commas  | naive_pct |  pct  | zero_padded
--------------+-----------+-------+-------------
 1,234,567.00 | 0.07%     | 7.25% |  00042
(1 row)
```

`%` in the pattern is printed literally — it does *not* multiply by 100, so do that yourself. Without `FM`, `to_char` reserves a leading space for the sign (see `' 00042'`). Format only at the very end: the result is text and no longer sorts or sums as a number.

Real use on the sample data

```sql
SELECT order_id, product_id,
       quantity * unit_price * (1 - discount)           AS exact,
       round(quantity * unit_price * (1 - discount), 2)  AS rounded
FROM order_items WHERE order_id IN (104, 107, 109) ORDER BY order_id, product_id;

-- Salary per month, rounded up to the next hundred
SELECT first_name, salary, ceil(salary / 12 / 100) * 100 AS monthly_rounded_up
FROM employees WHERE emp_id <= 4 ORDER BY emp_id;
```

Output:

```
 order_id | product_id |  exact   | rounded
----------+------------+----------+---------
      104 |          5 | 479.0000 |  479.00
      104 |          6 | 332.4905 |  332.49
      107 |          6 | 629.9820 |  629.98
      109 |          4 | 151.3000 |  151.30
(4 rows)

 first_name |  salary   | monthly_rounded_up
------------+-----------+--------------------
 Asha       | 250000.00 |              20900
 Vikram     | 185000.00 |              15500
 Neha       | 142000.00 |              11900
 Rahul      |  98000.00 |               8200
(4 rows)
```

The "divide, round to a unit, multiply back" trick (`ceil(x / 100) * 100`) works for any step size — to the nearest 5 minutes, 1,000 rupees, and so on.

Dialect map (numbers)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Truncate decimals	`TRUNCATE(x, d)`	`trunc(x, d)`	`ROUND(x, d, 1)`	— (cast tricks)
Random 0–1	`RAND()`	`random()`	`RAND()`	`random()` returns a 64-bit integer
Natural log	`LOG(x)` or `LN(x)`	`ln(x)`	`LOG(x)`	`ln(x)` (3.35+ math functions)
Base-10 log	`LOG10(x)`	`log(x)` / `log10(x)`	`LOG10(x)`	`log(x)` / `log10(x)`
Power	`POWER(a,b)` / `POW`	`power(a,b)` / `a ^ b`	`POWER(a,b)` (`^` is XOR, a bit-flipping operator!)	`pow(a,b)`
Number with commas	`FORMAT(x, 2)`	`to_char(x, 'FM999,999,990.00')`	`FORMAT(x, 'N2')`	no built-in (`printf('%,d', x)` groups integers only)
Convert type	`CONVERT(x, DECIMAL(5,2))` / `CAST`	`CAST(x AS numeric(5,2))` / `x::numeric(5,2)`	`CONVERT(DECIMAL(5,2), x)` / `CAST`	`CAST(x AS REAL)`
Integer division of ints	`DIV`	`/` or `div()`	`/`	`/`

Common mistakes

- Using `log()` expecting natural log in PostgreSQL.
- Formatting (`to_char`) inside calculations or before sorting.
- `round()` on `double precision` expecting "half up".

Self-check

1. *`floor(-2.5)`, `ceil(-2.5)`, `trunc(-2.5)`?* → -3, -2, -2.
2. *`round(123.456, -1)`?* → 120.
3. *PostgreSQL function for MySQL's `LOG(x)`?* → `ln(x)`.

TL;DR

`round/trunc/ceil/floor` for rounding (watch negatives and halves), `power/sqrt/exp/ln/log` for maths (PostgreSQL `log` is base 10), `mod` / `%` for remainders, `random` + `setseed` for repeatable randomness, `to_char` for display only.

3.3 String functions

What it is

String (text) functions join, cut, measure, clean and search text. They are the tools for turning messy input — stray spaces, mixed case, combined fields — into consistent values.

The functions

Function (PostgreSQL)	Does	Example → result
`concat(a, b, …)` / `a || b`	join strings	`concat('Hello', ' ', 'World')` → `Hello World`
`concat_ws(sep, a, b, …)`	join with a separator, skipping NULLs	`concat_ws('-', '2024', '01', '01')` → `2024-01-01`
`substring(s FROM start FOR len)` / `substr(s, start, len)`	part of a string (positions start at 1)	`substring('SQL Functions' FROM 1 FOR 3)` → `SQL`
`left(s, n)`, `right(s, n)`	first / last *n* characters (negative *n* = all but)	`left('Database', 3)` → `Dat`
`length(s)` / `char_length(s)`	number of characters	`length('Data')` → 4
`octet_length(s)`	number of bytes	`octet_length('Data')` → 4
`upper(s)`, `lower(s)`, `initcap(s)`	change case	`initcap('desk lamp')` → `Desk Lamp`
`replace(s, from, to)`	replace every occurrence of a substring	`replace('I love SQL', 'SQL', 'Postgres')`
`translate(s, from, to)`	replace character by character	`translate('2025/01/03', '/', '-')`
`trim(s)`, `ltrim`, `rtrim`, `trim(BOTH 'x' FROM s)`	remove spaces (or given characters) from ends	`trim(' SQL ')` → `SQL`
`position(sub IN s)` / `strpos(s, sub)`	1-based position of first match, 0 if absent	`strpos('Hello World', 'World')` → 7
`lpad(s, n, fill)`, `rpad(s, n, fill)`	pad (or cut) to length *n*	`lpad('7', 3, '0')` → `007`
`repeat(s, n)`, `reverse(s)`	repeat / reverse	`repeat('La', 3)` → `LaLaLa`
`split_part(s, delim, n)`	*n*-th field of a delimited string (fields separated by a marker such as `.` or `,`) (negative *n* from the end, PostgreSQL 14+)	`split_part('a@b.com', '@', 2)` → `b.com`
`format(fmt, …)`	printf-style building (fill `%s` slots in a template); `%I` quotes identifiers (table/column names), `%L` literals	
`regexp_replace`, `regexp_match`	regular-expression replace / capture	

How it works — concatenation and NULL

```sql
-- code/03_string_functions.sql
SELECT concat('Hello', ' ', 'World') AS concat, 'Hello' || ' ' || 'World' AS pipes,
       concat_ws('-', '2024', '01', '01') AS concat_ws;
SELECT concat('a', NULL, 'b') AS concat_null, 'a' || NULL || 'b' AS pipes_null,
       concat_ws(', ', 'Pune', NULL, 'India') AS ws_null;
```

Output:

```
   concat    |    pipes    | concat_ws
-------------+-------------+------------
 Hello World | Hello World | 2024-01-01
(1 row)

 concat_null | pipes_null |   ws_null
-------------+------------+-------------
 ab          | (null)     | Pune, India
(1 row)
```

In PostgreSQL `concat()` **ignores** NULL arguments, while `||` returns NULL if any part is NULL. `concat_ws` skips NULLs, so it's ideal for addresses with optional parts. **MySQL differs:** its `CONCAT()` returns NULL if any argument is NULL (its `CONCAT_WS` skips them).

Extracting parts

```
SELECT substring('SQL Functions' FROM 1 FOR 3) AS std_form, substring('SQL Functions', 5) AS from_5,
       substr('SQL Functions', 5, 4) AS substr, left('Database', 3) AS left_3,
       right('Postgres', 3) AS right_3, left('Database', -2) AS left_neg;
```

Output:

```
 std_form |  from_5   | substr | left_3 | right_3 | left_neg
----------+-----------+--------+--------+---------+----------
 SQL      | Functions | Func   | Dat    | res     | Databa
(1 row)
```

String positions in SQL start at **1**, not 0.

Measuring

```
SELECT length('Data') AS length, char_length('Data') AS char_length, octet_length('Data') AS bytes,
       length('  padded  ') AS with_spaces;
```

Output:

```
 length | char_length | bytes | with_spaces
--------+-------------+-------+-------------
      4 |           4 |     4 |          10
(1 row)
```

For plain ASCII (the 128 basic English letters, digits and symbols, one byte each), characters and bytes are the same. In a UTF-8 database (UTF-8 is the universal character encoding — the rule for storing any language's characters as bytes, using 1 to 4 bytes each) — which is what the book's runner creates (`ENCODING 'UTF8'`, `C` collation) — `é` counts as one character yet occupies two bytes:

```
SELECT current_setting('server_encoding') AS encoding, char_length('café') AS chars,
       octet_length('café') AS bytes, upper('café') AS upper_c_collation,
       upper('café' COLLATE "en-x-icu") AS upper_icu;
```

Output:

```
 encoding | chars | bytes | upper_c_collation | upper_icu
----------+-------+-------+-------------------+-----------
 UTF8     |     4 |     5 | CAFé              | CAFÉ
(1 row)
```

Notice the last two columns: case conversion follows the **collation**, and the plain `C` collation only knows the ASCII letters, so `é` stays lower case. A language-aware collation (here the ICU collation `en-x-icu`, available when PostgreSQL is built with ICU — International Components for Unicode, a widely used library of language rules) converts it properly. (A database created with the legacy `SQL_ASCII` encoding would not understand multibyte characters (characters stored in more than one byte) at all and would report 5 characters — always create databases as UTF-8.)

Portability trap: MySQL's `LENGTH()` counts **bytes** (use `CHAR_LENGTH()` for characters); SQL Server's `LEN()` counts characters but ignores trailing spaces (`DATALENGTH()` gives bytes).

Changing case, replacing, repeating

```
SELECT upper('hello') AS upper, lower('WORLD') AS lower, initcap('noise-cancelling headphones') AS initcap;
SELECT replace('I love SQL', 'SQL', 'Postgres') AS replace, translate('2025/01/03', '/', '-') AS translate,
       reverse('stressed') AS reverse, repeat('La', 3) AS repeat;
```

Output:

```
 upper | lower |           initcap
-------+-------+----------------------------
 HELLO | world | Noise-Cancelling Headphones
(1 row)

     replace     | translate  | reverse  | repeat
-----------------+------------+----------+--------
 I love Postgres | 2025-01-03 | desserts | LaLaLa
(1 row)
```

`upper`/`lower` are the standard way to compare case-insensitively: `WHERE lower(email) = lower(:input)` (`:input` stands for a value the application supplies). `replace` swaps whole substrings; `translate` maps single characters one-to-one.

Trimming

```
SELECT '[' || trim('   SQL   ') || ']' AS trim, '[' || ltrim('   SQL   ') || ']' AS ltrim,
       '[' || rtrim('   SQL   ') || ']' AS rtrim, trim(BOTH '*' FROM '**SQL**') AS trim_chars;
```

Output:

```
 trim  |  ltrim   |  rtrim   | trim_chars
-------+----------+----------+------------
 [SQL] | [SQL   ] | [   SQL] | SQL
(1 row)
```

(The brackets make the invisible spaces visible.) `trim` removes only leading and trailing characters, never those in the middle.

Searching and padding

```
SELECT position('abc' IN 'xxabcdef') AS position, strpos('Hello World', 'World') AS strpos,
       position('zzz' IN 'Hello') AS not_found;
SELECT lpad('SQL', 5, '*') AS lpad, rpad('SQL', 5, '*') AS rpad, lpad('7', 3, '0') AS zero_pad,
       lpad('Postgres', 3) AS lpad_truncates;
```

Output:

```
 position | strpos | not_found
----------+--------+-----------
        3 |      7 |         0
(1 row)

 lpad  | rpad  | zero_pad | lpad_truncates
-------+-------+----------+----------------
 **SQL | SQL** | 007      | Pos
(1 row)
```

`lpad` both pads and **truncates** to the requested length — careful with long values.

Splitting delimited text

```
SELECT split_part('www.example.com', '.', 1) AS first, split_part('www.example.com', '.', -1) AS last,
       array_to_string((string_to_array('www.example.com', '.'))[1:2], '.') AS first_two;
```

Output:

```
 first | last |  first_two
-------+------+-------------
 www   | com  | www.example
(1 row)
```

`split_part` replaces MySQL's `SUBSTRING_INDEX`: `SUBSTRING_INDEX(s, '.', -1)` (text after the last dot) is `split_part(s, '.', -1)`; "everything before the second dot" (`SUBSTRING_INDEX(s, '.', 2)`) needs the array form shown (an **array** is an ordered list of values held in one cell; `[1:2]` takes its first two elements). (Splitting into *rows* uses `string_to_table` / `unnest`, chapter 14.)

format() and regular expressions

```
SELECT format('%s earns %s', first_name, salary) AS sentence,
       format('SELECT * FROM %I WHERE name = %L', 'my table', 'O''Brien') AS safe_sql
FROM employees WHERE emp_id = 7;
SELECT regexp_replace('(022) 555-0199', '[^0-9]', '', 'g') AS digits_only,
       (regexp_match('order #A-1042 shipped', '#([A-Z])-(\d+)'))[2] AS order_number;
```

Output:

```
       sentence       |                    safe_sql
----------------------+-------------------------------------------------
 Priya earns 72000.00 | SELECT * FROM "my table" WHERE name = 'O''Brien'
(1 row)

 digits_only | order_number
-------------+--------------
 0225550199  | 1042
(1 row)
```

`%I` and `%L` quote an identifier and a literal safely — the basis of safe dynamic SQL (SQL text built as a string while the program runs, then executed; quoting stops *SQL injection*, where crafted input changes the meaning of the query — chapter 14). `regexp_replace(…, 'g')` replaces all matches (`g` = global); `regexp_match` returns an array of captured groups (the pieces of text matched by each bracketed part of the pattern), and `[2]` picks the second.

Cleaning real data

Our two Anita Shah rows differ only in formatting. Normalising them shows they're the same person:

```
SELECT customer_id, email AS raw, lower(trim(email)) AS clean, initcap(city) AS city_fixed
FROM customers WHERE last_name = 'Shah' ORDER BY customer_id;
```

Output:

```
 customer_id |           raw             |          clean          | city_fixed
-------------+---------------------------+-------------------------+------------
           1 | anita.shah@mail.example   | anita.shah@mail.example | Mumbai
           8 |   Anita.Shah@Mail.example | anita.shah@mail.example | Mumbai
(2 rows)
```

Better still, clean on the way *in* (application validation, a `CHECK` constraint — a rule the database tests on every insert — or a unique index on `lower(email)`, which refuses a second row with the same value — chapter 12).

Full names and email domains:

```sql
SELECT first_name || ' ' || last_name AS full_name,
       split_part(email, '@', 2) AS domain
FROM employees WHERE emp_id IN (1, 3, 5) ORDER BY emp_id;
```

Output:

```
 full_name  |     domain
------------+-----------------
 Asha Rao   | company.example
 Neha Gupta | Company.example
 Meera Iyer | (null)
(3 rows)
```

Meera has no email, so the domain is NULL — functions given NULL generally return NULL.

Dialect map (strings)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Concatenate	`CONCAT(a,b)` (NULL if any NULL)	`a || b`, `concat()` (skips NULL)	`a + b`, `CONCAT()` (NULL → '')	`a || b`, `concat()` (3.44+)
Characters	`CHAR_LENGTH(s)`	`length(s)` / `char_length(s)`	`LEN(s)` (no trailing spaces)	`length(s)`
Bytes	`LENGTH(s)`	`octet_length(s)`	`DATALENGTH(s)`	`length(CAST(s AS BLOB))` (BLOB = raw binary data)
Find position	`LOCATE(sub, s)`, `INSTR(s, sub)`, `POSITION(sub IN s)`	`position(sub IN s)`, `strpos(s, sub)`	`CHARINDEX(sub, s)`	`instr(s, sub)`
Split field	`SUBSTRING_INDEX(s, d, n)`	`split_part(s, d, n)`	`STRING_SPLIT` (rows) / `PARSENAME`	— (use `instr` + `substr`)
Left/right	`LEFT`, `RIGHT`	`left`, `right`	`LEFT`, `RIGHT`	`substr(s, 1, n)`, `substr(s, -n)`
Pad	`LPAD`, `RPAD`	`lpad`, `rpad`	`RIGHT(REPLICATE('0', n) + s, n)`	`printf('%05d', n)`
Repeat	`REPEAT`	`repeat`	`REPLICATE`	—
Regex replace	`REGEXP_REPLACE` (8.0+)	`regexp_replace`	`REGEXP_REPLACE` (SQL Server 2025)	extension

Common mistakes

- Assuming positions start at 0.
- Using `||` when a part may be NULL and getting a NULL result; use `concat`/`concat_ws` or `COALESCE`.
- Counting bytes instead of characters (MySQL `LENGTH`).
- Comparing case-insensitively without `lower()`/`ILIKE` in PostgreSQL.

Self-check

1. *Get the text after the `@` in an email in PostgreSQL?* → `split_part(email, '@', 2)`.
2. *`'a' || NULL` vs `concat('a', NULL)`?* → NULL vs `'a'`.
3. *Pad an order number 42 to `00042`?* → `lpad(42::text, 5, '0')` (or `to_char(42, 'FM00000')`).

TL;DR

`||`/`concat`/`concat_ws` join (mind NULLs), `substring`/`left`/`right`/`split_part` cut, `length`/`octet_length` measure, `lower`/`upper`/`initcap`/`trim`/`replace` clean, `position`/`strpos` search, `lpad`/`rpad` pad, `format` and `regexp_*` for power tasks.

3.4 Date and time functions

What it is

Time values cause more subtle SQL bugs than any other kind of data: months have different lengths, time zones shift, and "now" depends on when you ask. PostgreSQL has a rich, precise set of types and functions for them.

The types

Type	Holds	Example
`date`	a calendar date	`2025-03-18`
`time`	a time of day	`19:30:00`
`timestamp` (without time zone)	date + time, *no* zone — a wall-clock reading	`2025-03-18 19:30:00`
`timestamptz` (with time zone)	an absolute **instant** (one exact moment, the same everywhere on Earth); stored in UTC, shown in the session's time zone	`2025-03-18 14:00:00+00`
`interval`	a duration	`3 days`, `1 mon`, `02:30:00`

Guideline: use `timestamptz` for "when did it happen" (events, orders, logins) and `date` for calendar facts (birthdays, hire dates). Our sample `orders.placed_at` is a plain `timestamp` to keep outputs identical on every machine.

"Now"

A **transaction** is a group of statements that succeed or fail together, started with `BEGIN` and ended with `COMMIT` (a single statement on its own is also a transaction). Several "now" functions freeze at the transaction's start:

Function	Returns	Changes within a transaction?
`current_date`	today's `date`	no
`current_time`	time of day with zone (`timetz`)	no
`localtime`	time of day without zone	no
`now()` = `current_timestamp` = `transaction_timestamp()`	`timestamptz` at transaction start	**no**
`localtimestamp`	`timestamp` at transaction start	no
`statement_timestamp()`	start of the current statement	per statement
`clock_timestamp()`	the actual current time	**yes, every call**

```
-- code/04_date_functions.sql (the file sets SET TIME ZONE 'UTC' first)
SELECT current_date AS today, current_time AS time_with_tz, localtime AS local_time,
       now() AS now_tstz, localtimestamp AS local_ts;
```

Output (from the run that produced this book; yours will show your current time):

```
   today    |    time_with_tz    |   local_time    |           now_tstz            |          local_ts
------------+--------------------+-----------------+-------------------------------+---------------------------
 2026-09-28 | 19:15:38.702642+00 | 19:15:38.702642 | 2026-09-28 19:15:38.702642+00 | 2026-09-28 19:15:38.702642
(1 row)
```

`now()` is frozen for the whole transaction, so every row inserted in one transaction gets the same timestamp — usually what you want. `clock_timestamp()` keeps moving:

```sql
BEGIN;
SELECT now() AS tx_start, clock_timestamp() AS wall_clock;
SELECT pg_sleep(0.2);
SELECT now() AS tx_start_again, clock_timestamp() AS wall_clock_later;
COMMIT;
```

Output:

```
           tx_start            |          wall_clock
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.703095+00
(1 row)

        tx_start_again         |       wall_clock_later
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.905532+00
(1 row)
```

Taking the date part of a timestamp

MySQL's `DATE(ts)` is a cast in PostgreSQL:

```sql
SELECT placed_at, placed_at::date AS just_date, date_trunc('day', placed_at) AS truncated
FROM orders WHERE order_id = 103;
```

Output:

```
      placed_at      | just_date  |      truncated
---------------------+------------+---------------------
 2025-01-31 23:55:00 | 2025-01-31 | 2025-01-31 00:00:00
(1 row)
```

`::date` returns a `date`; `date_trunc('day', …)` returns a timestamp at midnight.

Date arithmetic

Expression	Result type	Meaning
`date + integer`	date	add days
`date - date`	integer	days between
`date/timestamp ± interval`	timestamp	add/subtract a duration
`timestamp - timestamp`	interval	elapsed time
`age(later, earlier)`	interval	"years mons days" difference

```sql
SELECT date '2024-01-01' + 3 AS plus_3_days, date '2024-01-01' + interval '3 days' AS plus_interval,
       date '2024-01-01' - interval '1 month' AS minus_month, date '2024-01-05' - date '2024-01-01' AS
days_between;
```

Output:

```
 plus_3_days |    plus_interval    |     minus_month     | days_between
-------------+---------------------+---------------------+--------------
 2024-01-04  | 2024-01-04 00:00:00 | 2023-12-01 00:00:00 |            4
(1 row)
```

Month arithmetic clamps to the last valid day, which makes it non-reversible:

```
SELECT date '2025-01-31' + interval '1 month' AS jan31_plus_month,
       date '2024-02-29' + interval '1 year' AS leap_plus_year,
       date '2025-01-31' + interval '1 month' - interval '1 month' AS round_trip;
```

Output:

```
  jan31_plus_month   |   leap_plus_year    |     round_trip
---------------------+---------------------+---------------------
 2025-02-28 00:00:00 | 2025-02-28 00:00:00 | 2025-01-28 00:00:00
(1 row)
```

31 January + 1 month = 28 February, and going back a month lands on 28 January, not the 31st. Billing systems must decide explicitly what "one month later" means.

Extracting fields: EXTRACT and date_part

`extract(field FROM value)` is standard SQL; `date_part('field', value)` is PostgreSQL's function form. Since PostgreSQL 14, `extract` returns `numeric`.

```
SELECT extract(YEAR FROM date '2024-01-01') AS year, extract(MONTH FROM date '2024-01-01') AS month,
       extract(DAY FROM date '2024-01-01') AS day, extract(DOW FROM date '2024-01-01') AS dow_0_sun,
       extract(ISODOW FROM date '2024-01-01') AS isodow_1_mon, extract(QUARTER FROM date '2024-08-15') AS quarter,
       date_part('week', date '2024-01-01') AS iso_week;
```

Output:

```
 year | month | day | dow_0_sun | isodow_1_mon | quarter | iso_week
------+-------+-----+-----------+--------------+---------+----------
 2024 |     1 |   1 |         1 |            1 |       3 |        1
(1 row)
```

1 January 2024 was a **Monday**: `DOW` numbers Sunday 0 … Saturday 6, `ISODOW` numbers Monday 1 … Sunday 7. Other useful fields: `HOUR`, `MINUTE`, `SECOND`, `DOY` (day of year), `EPOCH` (seconds since 1970-01-01 UTC).

Truncating to a period: date_trunc

`date_trunc(unit, ts)` rounds *down* to the start of the unit — the workhorse of time-series reports (reports of values over time, such as "sales per month"):

```
SELECT placed_at, date_trunc('month', placed_at) AS month_start, date_trunc('week', placed_at) AS week_start,
       date_trunc('hour', placed_at) AS hour_start
FROM orders WHERE order_id = 107;
```

Output:

```
      placed_at      |     month_start     |     week_start      |     hour_start
---------------------+---------------------+---------------------+---------------------
 2025-03-18 19:30:00 | 2025-03-01 00:00:00 | 2025-03-17 00:00:00 | 2025-03-18 19:00:00
(1 row)
```

Weeks start on **Monday** (ISO 8601, the international standard for writing dates and times). Grouping by `date_trunc('month', …)` gives one sortable key per month — see exercise E5 in 3.6.

Formatting and parsing

`to_char(value, pattern)` formats; `to_date` / `to_timestamp` parse; `make_date` / `make_timestamp` build from numbers.

Pattern	Meaning	Example
`YYYY`, `MM`, `DD`	year, month number, day	`2025`, `03`, `18`
`Month`, `Mon`	month name, abbreviation	`March`, `Mar`
`Day`, `Dy`	weekday name, abbreviation	`Tuesday`, `Tue`
`HH24`, `HH12`, `MI`, `SS`, `AM`	hour, minute, second, meridiem	`19`, `07`, `30`, `00`, `PM`
`Q`	quarter	`1`
`FM` prefix	drop padding spaces/zeros	`FMDay` → `Monday`
`"text"`	literal text	`"Q"`

```
SELECT to_char(date '2024-01-01', 'FMDay, FMMonth FMDD, YYYY') AS long_form,
       to_char(timestamp '2025-03-18 19:30', 'DD Mon YYYY HH12:MI AM') AS short_form,
       to_char(timestamp '2025-03-18 19:30', 'YYYY-"Q"Q') AS quarter_label;
SELECT to_date('18/03/2025', 'DD/MM/YYYY') AS parsed_date,
       to_timestamp('2025-03-18 19:30', 'YYYY-MM-DD HH24:MI') AS parsed_ts,
       make_date(2025, 3, 18) AS made_date;
```

Output:

```
        long_form        |      short_form      | quarter_label
-------------------------+----------------------+---------------
 Monday, January 1, 2024 | 18 Mar 2025 07:30 PM | 2025-Q1
(1 row)

 parsed_date |       parsed_ts        | made_date
-------------+------------------------+------------
 2025-03-18  | 2025-03-18 19:30:00+00 | 2025-03-18
(1 row)
```

`to_timestamp` returns a `timestamptz` interpreted in the session time zone (UTC here). For ISO strings (`'2025-03-18'`) a plain cast is simpler; use `to_date` for other layouts like `18/03/2025`, which is ambiguous otherwise.

Differences between moments

```
SELECT order_id, placed_at, shipped_at,
       shipped_at - placed_at AS time_to_ship,
       round(extract(EPOCH FROM shipped_at - placed_at) / 3600, 1) AS hours_to_ship,
       shipped_at::date - placed_at::date AS calendar_days
FROM orders WHERE shipped_at IS NOT NULL AND order_id <= 104 ORDER BY order_id;
```

Output:

```
 order_id |      placed_at      |     shipped_at      |  time_to_ship  | hours_to_ship | calendar_days
----------+---------------------+---------------------+----------------+---------------+---------------
      101 | 2025-01-03 10:15:00 | 2025-01-04 09:00:00 | 22:45:00       |          22.8 |             1
      102 | 2025-01-17 22:40:00 | 2025-01-19 11:30:00 | 1 day 12:50:00 |          36.8 |             2
      103 | 2025-01-31 23:55:00 | 2025-02-02 08:10:00 | 1 day 08:15:00 |          32.3 |             2
      104 | 2025-02-11 09:05:00 | 2025-02-12 16:45:00 | 1 day 07:40:00 |          31.7 |             1
(4 rows)
```

Three different answers to "how long did shipping take?": an `interval`, a number of hours (via `EPOCH` seconds ÷ 3600), and the number of calendar-date boundaries crossed. Order 101 took under a day but crossed one midnight. Decide which one the business means.

`age()` gives a human-style difference:

```
SELECT first_name, hire_date, age(date '2025-07-01', hire_date) AS tenure_on_2025_07_01
FROM employees WHERE emp_id IN (1, 9, 15) ORDER BY emp_id;
```

Output:

```
 first_name | hire_date  |  tenure_on_2025_07_01
------------+------------+------------------------
 Asha       | 2015-01-05 | 10 years 5 mons 27 days
 Sara       | 2024-04-01 | 1 year 3 mons
 Rohan      | 2025-06-02 | 29 days
(3 rows)
```

`age(x)` with one argument measures from `current_date` at midnight. Whole years of service: `extract(YEAR FROM age(current_date, hire_date))`.

Expected delivery ten days after ordering:

```
SELECT order_id, placed_at::date AS order_date, (placed_at + interval '10 days')::date AS expected_by
FROM orders WHERE order_id IN (101, 105, 113) ORDER BY order_id;
```

Output:

```
 order_id | order_date | expected_by
----------+------------+-------------
      101 | 2025-01-03 | 2025-01-13
      105 | 2025-02-28 | 2025-03-10
      113 | 2025-06-02 | 2025-06-12
(3 rows)
```

Time zones

A `timestamptz` is one instant; how it *prints* depends on the session's `TimeZone` setting. `AT TIME ZONE` converts between instants and wall-clock readings:

Expression	Gives
`timestamptz AT TIME ZONE 'Zone'`	the wall-clock `timestamp` in that zone at that instant
`timestamp AT TIME ZONE 'Zone'`	the `timestamptz` instant when that zone's clock showed that reading

```
SET TIME ZONE 'UTC';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_utc;
SET TIME ZONE 'Asia/Kolkata';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_ist,
       timestamptz '2025-03-18 19:30:00+05:30' AT TIME ZONE 'America/New_York' AS wall_clock_ny,
       timestamp '2025-03-18 19:30' AT TIME ZONE 'Europe/London' AS london_wall_clock_as_instant;
```

Output:

```
      shown_in_utc
------------------------
 2025-03-18 14:00:00+00
(1 row)

       shown_in_ist        |    wall_clock_ny    | london_wall_clock_as_instant
---------------------------+---------------------+------------------------------
 2025-03-18 19:30:00+05:30 | 2025-03-18 10:00:00 | 2025-03-19 01:00:00+05:30
(1 row)
```

7:30 pm in India is 14:00 UTC and 10:00 in New York (already on daylight-saving time on 18 March). 7:30 pm in London (still on GMT, Greenwich Mean Time — the UK's winter clock, equal to UTC) is 01:00 the next day in India. Use named zones (`'Asia/Kolkata'`), not fixed offsets, so daylight-saving rules are applied.

A calendar of dates

`generate_series` produces rows — here, one per day — which is how you build calendars or fill gaps in reports (chapter 7):

```
SELECT d::date AS day, to_char(d, 'Dy') AS weekday
FROM generate_series(date '2025-03-01', date '2025-03-05', interval '1 day') AS d;
```

Output:

```
    day     | weekday
------------+---------
 2025-03-01 | Sat
 2025-03-02 | Sun
 2025-03-03 | Mon
 2025-03-04 | Tue
 2025-03-05 | Wed
(5 rows)
```

Dialect map (dates)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Now	`NOW()`	`now()`	`SYSDATETIME()` / `GETDATE()`	`datetime('now')`
Today	`CURRENT_DATE` / `CURDATE()`	`current_date`	`CAST(GETDATE() AS date)`	`date('now')`
Date part of datetime	`DATE(ts)`	`ts::date`	`CAST(ts AS date)`	`date(ts)`
Add 3 days	`DATE_ADD(d, INTERVAL 3 DAY)`	`d + interval '3 days'` (or `d + 3` for a date)	`DATEADD(day, 3, d)`	`date(d, '+3 days')`
Subtract a month	`DATE_SUB(d, INTERVAL 1 MONTH)`	`d - interval '1 month'`	`DATEADD(month, -1, d)`	`date(d, '-1 month')`
Year / month / day	`YEAR(d)`, `MONTH(d)`, `DAY(d)`, `EXTRACT`	`extract(YEAR FROM d)` …	`YEAR(d)`, `DATEPART(month, d)`	`strftime('%Y', d)`
Format	`DATE_FORMAT(d, '%W, %M %e, %Y')`	`to_char(d, 'FMDay, FMMonth FMDD, YYYY')`	`FORMAT(d, 'dddd, MMMM d, yyyy')`	`strftime(…)`
Days between	`DATEDIFF(end, start)`	`end::date - start::date`	`DATEDIFF(day, start, end)`	`julianday(end) - julianday(start)`
Difference in a unit	`TIMESTAMPDIFF(HOUR, start, end)`	`extract(EPOCH FROM end - start) / 3600`	`DATEDIFF(hour, start, end)`	`(julianday(end) - julianday(start)) * 24`
Truncate to month	`DATE_FORMAT(d, '%Y-%m-01')`	`date_trunc('month', d)`	`DATETRUNC(month, d)` (2022+)	`date(d, 'start of month')`

Note the argument orders: MySQL `DATEDIFF(end, start)` vs SQL Server `DATEDIFF(unit, start, end)`. SQL Server's `DATEDIFF` counts *boundaries crossed* (like `calendar_days` above), not elapsed time.

Common mistakes

- Comparing a timestamp column to a date-only upper bound (see chapter 2, 2.4) — use half-open ranges.
- Applying a function to an indexed column (one with an **index** — a sorted lookup structure that lets the database find rows without reading them all) in `WHERE` (`WHERE date_trunc('month', placed_at) = …`) — it can stop the index being used; rewrite as a range (chapter 12).
- Storing local times in `timestamp` for a multi-time-zone app — use `timestamptz`.
- Assuming "+1 month" can be undone with "−1 month".

In the interview

> **30-second answer:** "I store instants as timestamptz, truncate with date_trunc for per-day or per-month reports, filter with half-open ranges so indexes still work, and compute durations with timestamp subtraction or EXTRACT(EPOCH …). I'm careful that month arithmetic clamps and that now() is fixed per transaction."

- *Orders per month including months with zero orders?* → Generate the months with `generate_series` and left-join the counts (a *left join* keeps every row of the first table even when the second has no match; chapters 5 and 7).
- *Difference between `timestamp` and `timestamptz`?* → `timestamptz` is an absolute instant converted to/from the session zone; `timestamp` is a zone-less wall-clock reading.

Self-check

1. *`date '2025-03-31' - interval '1 month'`?* → `2025-02-28 00:00:00`.
2. *First day of the month of `placed_at`?* → `date_trunc('month', placed_at)`.
3. *Why might two `now()` calls in one transaction be equal?* → `now()` returns the transaction start time.
4. *Hours between two timestamps as a number?* → `extract(EPOCH FROM b - a) / 3600`.

TL;DR

`date`, `timestamp`, `timestamptz`, `interval`. `now()` (transaction start) vs `clock_timestamp()`. Add intervals; subtract dates for days; `extract` fields; `date_trunc` to periods; `to_char`/`to_date` to format/parse; `AT TIME ZONE` converts; month maths clamps.

3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST

What it is

Conditional expressions put *if-then-else* logic inside a query. They are expressions, so they return a value and can be used wherever a column can.

CASE

Searched CASE tests conditions in order and returns the result of the first true one:

```sql
-- code/05_case_coalesce_nullif.sql
SELECT name, unit_price,
       CASE WHEN unit_price >= 300 THEN 'premium'
            WHEN unit_price >= 50  THEN 'standard'
            ELSE 'budget' END AS price_band
FROM products ORDER BY unit_price DESC;
```

Output:

```
            name             | unit_price | price_band
-----------------------------+------------+------------
 Laptop Pro 14               |    1299.00 | premium
 Standing Desk               |     499.00 | premium
 Ergonomic Chair             |     349.99 | premium
 Noise-Cancelling Headphones |     199.95 | standard
 Mechanical Keyboard         |      89.00 | standard
 Gift Card                   |      50.00 | standard
 USB-C Hub                   |      49.50 | budget
 Desk Lamp                   |      35.00 | budget
 Wireless Mouse              |      24.99 | budget
(9 rows)
```

Order matters: a 1,299 laptop is also `>= 50`, but the first matching branch wins. Write branches from most to least specific.

Simple CASE compares one expression with a list of values:

```
SELECT order_id, status,
       CASE status WHEN 'pending' THEN 'Awaiting dispatch'
                   WHEN 'shipped' THEN 'On the way'
                   WHEN 'delivered' THEN 'Done'
       END AS customer_message
FROM orders WHERE order_id IN (101, 105, 107, 113) ORDER BY order_id;
```

Output:

```
 order_id |  status   | customer_message
----------+-----------+------------------
      101 | delivered | Done
      105 | cancelled | (null)
      107 | shipped   | On the way
      113 | pending   | Awaiting dispatch
(4 rows)
```

No `ELSE` means `ELSE NULL` — the cancelled order gets NULL. Always write an `ELSE` unless NULL is really what you want. Also, simple `CASE x WHEN NULL` never matches (it uses `=`); test NULLs with a searched `CASE WHEN x IS NULL`.

`CASE` appears everywhere: in `ORDER BY` for custom sort orders (chapter 2, 2.7), inside aggregates for conditional counts (`count(*) FILTER (…)` / `sum(CASE …)`, chapter 4), in `UPDATE … SET` for conditional changes (chapter 9), and for pivots (turning row values into columns, e.g. one column per month; chapter 14).

COALESCE — first non-NULL

`coalesce(a, b, c, …)` returns the first argument that isn't NULL (or NULL if all are). It's the standard way to supply defaults.

```
SELECT first_name, email, coalesce(email, 'no email on file') AS contact,
       commission_pct, coalesce(commission_pct, 0) AS commission_or_zero
FROM employees WHERE emp_id IN (5, 6, 9, 11) ORDER BY emp_id;
```

Output:

```
 first_name |             email             |            contact            | commission_pct | commission_or_zero
------------+-------------------------------+-------------------------------+----------------+--------------------
 Meera      | (null)                        | no email on file              |         (null) |                  0
 Daniel     | daniel.brooks@company.example | daniel.brooks@company.example |           0.05 |               0.05
 Sara       | sara.khan@company.example     | sara.khan@company.example     |         (null) |                  0
 Ananya     | (null)                        | no email on file              |         (null) |                  0
(4 rows)
```

Why it matters — NULL "poisons" arithmetic:

```
SELECT first_name, salary, commission_pct,
       salary * (1 + commission_pct)              AS naive_total,
       salary * (1 + coalesce(commission_pct, 0)) AS total
FROM employees WHERE dept_id = 2 ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | commission_pct | naive_total |    total
------------+-----------+----------------+-------------+-------------
 Daniel     | 150000.00 |           0.05 | 157500.0000 | 157500.0000
 Priya      |  72000.00 |           0.10 |  79200.0000 |  79200.0000
 Arjun      |  68000.00 |           0.10 |  74800.0000 |  74800.0000
 Sara       |  68000.00 |         (null) |      (null) |    68000.00
(4 rows)
```

Without `coalesce`, Sara's total pay is unknown. All arguments must share a compatible type — `coalesce(salary, 'n/a')` fails; convert first (`coalesce(salary::text, 'n/a')`).

NULLIF — turn a value into NULL

`nullif(a, b)` returns NULL if `a = b`, otherwise `a`. Two classic uses:

```
-- 1. Avoid division by zero
SELECT a, b, a / NULLIF(b, 0) AS safe_ratio
FROM (VALUES (10, 2), (10, 0)) AS t(a, b);

-- 2. Treat blank strings as missing
SELECT email, NULLIF(trim(email), '') AS email_or_null
FROM (VALUES ('x@y.example'), ('   '), (NULL)) AS t(email);
```

Output:

```
 a  | b | safe_ratio
----+---+------------
 10 | 2 |          5
 10 | 0 |     (null)
(2 rows)

    email    | email_or_null
-------------+---------------
 x@y.example | x@y.example
             | (null)
 (null)      | (null)
(3 rows)
```

`x / 0` raises *division by zero* and aborts the whole query; `x / NULLIF(0, 0)` is `x / NULL` = NULL. Combine them: `coalesce(a / nullif(b, 0), 0)` if a zero is more useful than NULL.

CASE with dates and NULLs together

```
SELECT name, start_date, end_date,
       coalesce(end_date, date '2025-12-31') AS planned_or_open_end,
       CASE WHEN end_date IS NULL THEN 'running' ELSE 'finished' END AS state
FROM projects ORDER BY project_id;
```

Output:

```
       name       | start_date |  end_date  | planned_or_open_end |  state
------------------+------------+------------+---------------------+----------
 Checkout Revamp  | 2024-09-01 | 2025-03-31 | 2025-03-31          | finished
 Mobile App       | 2025-01-15 | (null)     | 2025-12-31          | running
 Spring Campaign  | 2025-02-01 | 2025-04-30 | 2025-04-30          | finished
 Year-End Audit   | 2024-11-01 | 2025-01-31 | 2025-01-31          | finished
 Helpdesk Chatbot | 2025-04-01 | (null)     | 2025-12-31          | running
(5 rows)
```

GREATEST and LEAST

Return the largest / smallest of their arguments (not across rows — that's `max`/`min`, chapter 4). In PostgreSQL they **ignore NULLs**:

```
SELECT greatest(1, NULL, 3) AS greatest, least(1, NULL, 3) AS least;
```

```
 greatest | least
----------+-------
        3 |     1
(1 row)
```

MySQL and Oracle return NULL if any argument is NULL — wrap arguments in `coalesce` for portable code. Typical use: `greatest(price - discount, 0)` to floor a value at zero.

Dialect map (conditionals)

Task	Standard / PostgreSQL	MySQL	SQL Server	Oracle
If-then-else	`CASE WHEN … THEN … ELSE … END`	also `IF(cond, a, b)`	also `IIF(cond, a, b)`	also `DECODE(…)`
First non-NULL	`COALESCE(a, b, …)`	also `IFNULL(a, b)`	also `ISNULL(a, b)`	also `NVL(a, b)`
Value → NULL	`NULLIF(a, b)`	same	same	same
Greatest/least	`GREATEST`, `LEAST` (ignore NULLs)	same names (NULL if any NULL)	`GREATEST` / `LEAST` (2022+, ignore NULLs)	same names (NULL if any NULL)

Prefer the standard forms (`CASE`, `COALESCE`, `NULLIF`) — they work everywhere.

Common mistakes

- Forgetting `ELSE` and getting unexpected NULLs.
- Overlapping `WHEN` conditions in the wrong order.
- `CASE x WHEN NULL …` — never true.
- Mixing result types in branches (`THEN 1 ELSE 'none'`) — PostgreSQL errors; make them the same type.

In the interview

> **30-second answer:** "CASE is SQL's if-else and works anywhere an expression does — SELECT, ORDER BY, inside SUM for conditional aggregation. COALESCE returns the first non-null, so I use it for defaults and to stop NULL wiping out arithmetic. NULLIF turns a value into NULL, mainly to avoid division by zero."

- *Count delivered and cancelled orders in one row?* → `sum(CASE WHEN status='delivered' THEN 1 ELSE 0 END)`, or `count(*) FILTER (WHERE status='delivered')` in PostgreSQL (chapter 4).
- *`ISNULL` vs `COALESCE` in SQL Server?* → `ISNULL` takes two arguments and uses the first argument's type; `COALESCE` is standard, takes many, and uses type precedence (the rule for which type wins when arguments differ).

Self-check

1. *What does a CASE return when no branch matches and there's no ELSE?* → NULL.
2. *Safe average price per item when quantity may be 0?* → `total / NULLIF(quantity, 0)`.
3. *`coalesce(NULL, NULL, 'x', 'y')`?* → `'x'`.

TL;DR

`CASE` = if/else (first true branch wins; no `ELSE` → NULL). `COALESCE` = first non-NULL (defaults). `NULLIF` = value → NULL (safe division, blank → NULL). `GREATEST`/`LEAST` compare arguments (PostgreSQL skips NULLs).

3.6 Worked exercises

Try each before reading the solution. They combine this chapter with a preview of aggregates (`sum`, `count`, `GROUP BY` — chapter 4) and a simple join (`JOIN … USING` — a **join** pairs rows of two tables whose shared column, here `order_id`, matches; chapter 5).

E1. Total value of all delivered orders, rounded to 2 decimals and shown with thousands separators.

```sql
-- code/06_function_exercises.sql
SELECT to_char(round(sum(quantity * unit_price * (1 - discount)), 2), 'FM999,999,990.00') AS total_sales
FROM order_items oi
JOIN orders o USING (order_id)
WHERE o.status = 'delivered';
```

```
 total_sales
-------------
 4,508.11
(1 row)
```

Compute with numbers, round, and only then format — `to_char` is the last step.

E2. The square root of the number of order lines, and the natural log of total revenue.

```
SELECT round(sqrt(count(*))::numeric, 3) AS sqrt_lines,
       round(ln(sum(quantity * unit_price * (1 - discount))), 4) AS ln_revenue
FROM order_items;
```

```
 sqrt_lines | ln_revenue
------------+------------
      4.690 |     9.2130
(1 row)
```

`sqrt` of an integer returns `double precision`, and `round(x, d)` with a decimal-places argument needs `numeric` — hence the cast. Note `ln`, not `log`, for the natural logarithm.

E3. Customer display names like "Shah, A." and lower-case email domains.

```
SELECT customer_id,
       last_name || ', ' || left(first_name, 1) || '.' AS display_name,
       lower(split_part(trim(email), '@', 2))          AS domain
FROM customers ORDER BY customer_id LIMIT 5;
```

```
 customer_id | display_name |    domain
-------------+--------------+--------------
           1 | Shah, A.     | mail.example
           2 | Carter, B.   | mail.example
           3 | Li, C.       | mail.example
           4 | Menon, D.    | (null)
           5 | Walker, E.   | mail.example
(5 rows)
```

E4. Revenue per year and month, excluding cancelled orders.

```
SELECT extract(YEAR FROM o.placed_at)  AS year,
       extract(MONTH FROM o.placed_at) AS month,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1, 2
ORDER BY 1, 2;
```

```
 year | month | revenue
------+-------+---------
 2025 |     1 | 1871.90
 2025 |     2 |  811.49
 2025 |     3 |  804.95
 2025 |     4 | 2148.75
 2025 |     5 |  249.93
 2025 |     6 | 1516.60
(6 rows)
```

E5. The same idea with one sortable month key (orders per month):

```sql
SELECT to_char(date_trunc('month', o.placed_at), 'YYYY-MM') AS month,
       count(DISTINCT o.order_id) AS orders
FROM orders o
WHERE o.status <> 'cancelled'
GROUP BY date_trunc('month', o.placed_at)
ORDER BY date_trunc('month', o.placed_at);
```

```
  month  | orders
---------+--------
 2025-01 |      3
 2025-02 |      1
 2025-03 |      2
 2025-04 |      3
 2025-05 |      1
 2025-06 |      2
(6 rows)
```

Grouping by `date_trunc` keeps a real timestamp for sorting; `to_char` only dresses it up for display. The expected-delivery exercise (order date + 10 days) is in 3.4.

Chapter 3 TL;DR

- **Types:** everything has one; `numeric` for money, `text` for strings, `date`/`timestamptz` for time; convert with `CAST(x AS t)` or `x::t`; PostgreSQL refuses silent coercions.
- **Numbers:** `round`/`trunc`/`ceil`/`floor` (numeric rounds half away from zero, floats to even), `power`, `sqrt`, `exp`, `ln`, `log` (base 10 in PostgreSQL!), `mod`/`%`, `random` + `setseed`, `to_char` for display.
- **Strings:** `||` vs `concat` (NULL handling), `substring`/`left`/`right`/`split_part`, `length` vs `octet_length`, `lower`/`upper`/`initcap`, `trim`, `replace`/`translate`, `position`/`strpos`, `lpad`/`rpad`, `format`, `regexp_replace`.
- **Dates:** `now()` vs `clock_timestamp()`, `+ interval`, `date - date`, `age`, `extract`, `date_trunc`, `to_char`/`to_date`, `AT TIME ZONE`; month arithmetic clamps.
- **Conditionals:** `CASE` (no ELSE → NULL), `COALESCE` (defaults), `NULLIF` (safe division), `GREATEST`/`LEAST`.
- MySQL names differ a lot (`DATE_ADD`, `DATE_FORMAT`, `SUBSTRING_INDEX`, `IFNULL`, `LOG` = ln, `LENGTH` = bytes) — use the dialect maps.

Next: Chapter 4 — Aggregation.